2-4 同步与互斥

- 1. 同步关系
  - 基本含义
    - **形成原因**：进程并发执行时，由于共享数据或协作执行，会出现直接相互制约。
    - **典型场景**：输入进程向计算进程通过共享缓冲区提供数据。
    - **缓冲区满**：输入进程不能继续投放数据，需要阻塞。
    - **缓冲区空**：计算进程不能取出数据，需要阻塞。
    - **关系本质**：一个进程的推进依赖另一个进程到达某个位置或完成某个动作。
- 2. 互斥关系
  - 基本含义
    - **形成原因**：多个进程并发执行时，可能竞争同一个只能独占使用的资源。
    - **资源使用**：使用前先提出申请，由系统统一分配。
    - **关系本质**：多个进程之间存在间接相互制约。
    - **访问要求**：同一时刻只允许一个进程访问相关资源。
- 3. 临界资源
  - 定义
    - **临界资源**：一段时间内只允许一个进程访问的资源，也称独占资源。
    - **共享方式**：各进程之间必须采用互斥方式访问。
    - **硬件资源**：打印机等独占设备可以是临界资源。
    - **软件资源**：共享变量也可以是临界资源。
  - 共享变量问题
    - **问题来源**：高级语句可能被分解成多条机器指令。
    - **并发执行**：对同一个变量执行 `counter++` 和 `counter--` 时，中间结果可能互相覆盖。
    - **结果偏差**：初值为 `2` 时，执行一次加一和一次减一后，结果可能不是 `2` 。
    - **处理方式**：把共享变量 `counter` 当作临界资源，让进程互斥访问。
- 4. 临界区
  - 各部分含义
    - **进入区**：进入临界区前检查临界资源是否正在被访问。
    - **临界区**：每个进程中访问临界资源的那段代码。
    - **退出区**：离开临界区后恢复临界资源访问标志。
    - **剩余区**：进程中除进入区、临界区、退出区之外的其他代码。
- 5. 同步机制应遵循的规则
  - 空闲让进
    - **资源空闲**：没有进程处于临界区。
    - **进入要求**：应允许一个请求进入临界区的进程立即进入。
    - **目的**：有效利用临界资源。
  - 忙则等待
    - **资源忙碌**：已有进程进入临界区。
    - **等待要求**：其他试图进入临界区的进程必须等待。
    - **目的**：保证对临界资源的互斥访问。
  - 有限等待
    - **访问请求**：进程要求访问临界资源。
    - **等待限制**：应保证其在有限时间内进入自己的临界区。
    - **目的**：避免进程陷入死等状态。
  - 让权等待
    - **不能进入**：进程暂时不能进入自己的临界区。
    - **处理要求**：应立即释放处理机。
    - **目的**：避免进程陷入忙等状态。
- 6. 软件同步机制
  - 单标志法
    - **公共变量**：设置整型变量 `turn` ，指示允许进入临界区的进程编号。
    - **进入规则**： `turn = 0` 时允许 `P0` 进入， `turn = 1` 时允许 `P1` 进入
    - **退出规则**：进程退出临界区时，把 `turn` 改为另一个进程编号。
    - **主要问题**：两个进程必须交替进入临界区。
    - **违反规则**：若一个进程不再进入临界区，另一个进程也无法进入，违背空闲让进。
  - 双标志先检查法
    - **公共变量**：设置布尔数组 `flag[2]` ，标记各进程进入临界区的意愿。
    - **标志含义**： `flag[i] = true` 表示 `Pi` 想进入临界区。
    - **进入规则**：先检查对方是否想进入；若对方想进入则等待，否则把自己的 `flag[i]` 置为 `true` 后进入。
    - **退出规则**：退出临界区时，把自己的 `flag[i]` 置为 `false` 。
    - **主要问题**：检查对方标志和设置自己标志不能一次完成。
    - **可能结果**：两个进程都先看到对方不想进入，再分别设置自己的标志，从而同时进入临界区。
    - **违反规则**：可能违背忙则等待。
  - 双标志后检查法
    - **改进思路**：先设置自己的标志，再检查对方标志。
    - **进入规则**：把自己的 `flag[i]` 置为 `true` 后，若对方 `flag[j] = true` ，则等待；否则进入临界区。
    - **退出规则**：退出临界区时，把自己的 `flag[i]` 置为 `false` 。
    - **主要问题**：两个进程可能都先设置好自己的标志，再同时检查对方标志。
    - **可能结果**：双方都想进入临界区，却都被对方标志挡住。
    - **违反规则**：可能导致饥饿，违背有限等待。
  - Peterson 算法
    - **基本思想**：结合单标志法和双标志后检查法。
    - **flag[] 作用**：解决互斥访问问题。
    - **turn 作用**：解决饥饿问题。
    - **进入规则**：进程进入临界区前，先设置自己的 `flag` 标志，再设置允许进入的 `turn` 标志。
    - **判断规则**：同时检测对方的 `flag` 和 `turn` ，双方同时要求进入时只允许一个进程进入。
    - **礼让机制**：若双方都争着进入临界区，可让进程把进入机会让给对方。
    - **满足规则**：遵循空闲让进、忙则等待、有限等待。
    - **未满足规则**：未遵循让权等待，不能进入的进程仍可能在 `while` 中忙等。
- 7. 硬件同步机制
  - 基本作用
    - **引入原因**：软件方法解决互斥进入临界区有难度，也存在局限。
    - **解决方式**：计算机提供特殊硬件指令来支持互斥。
    - **核心特点**：相关硬件指令的执行过程不可分割。
  - 关中断
    - **基本方法**：进程在临界区执行期间关闭中断。
    - **直接效果**：不会引发调度，也就不会发生进程或线程切换。
    - **缺点一**：滥用关中断权力会导致严重后果。
    - **缺点二**：关中断时间过长会影响系统效率。
    - **缺点三**：不适用于多 CPU 系统；一个处理器关中断不能阻止其他处理器执行相同临界代码。
  - Test-and-Set 指令
    - **简称**：TS 指令。
    - **原子性**：可看作执行过程不可分割的函数过程，也就是原语。
    - **lock = FALSE**：表示资源空闲。
    - **lock = TRUE**：表示资源正在被使用。
    - **使用方式**：为每个临界资源设置一个 `lock` ，初值为 `FALSE` 。
    - **进入过程**：进程进入临界区前用 TS 测试 `lock` ；若为 `FALSE` ，则进入并把 `lock` 置为 `TRUE` 。
  - Swap 指令
    - **基本含义**：对换指令，用于交换两个字的内容。
    - **使用方式**：为每个临界资源设置一个全局布尔变量 `lock` ，初值为 `FALSE` 。
    - **实现目标**：利用硬件指令完成进程互斥。
    - **主要问题**：访问进程会不断测试，处于忙等状态。
    - **违反规则**：不符合让权等待。
- 8. 信号量机制
  - 整型信号量
    - **定义**：用于表示资源数目的整型量 `S` 。
    - **操作类型**：初始化、 `wait` 、 `signal` 。
    - **原子性**： `wait` 和 `signal` 均为原子操作，执行时不可中断。
    - **互斥修改**：一个进程修改某信号量时，其他进程不能同时修改该信号量。
    - **主要问题**：会对信号量不断测试，使进程处于忙等状态。
  - 记录型信号量
    - **基本作用**：一种不存在忙等现象的进程同步机制。
    - **value**：表示资源数目的整型变量。
    - **L**：进程链表，用于链接所有等待该资源的进程。
    - **wait 操作**：等价于申请资源。
    - **signal 操作**：等价于释放资源。
    - **称呼对应**： `wait(A) = P(A)` ， `signal(A) = V(A)` 。
  - 信号量实现进程互斥
    - **互斥信号量**：设置 `mutex = 1` 。
    - **代码位置**：把各进程访问临界资源的临界区放在 `wait(mutex)` 和 `signal(mutex)` 之间。
    - **进入临界区**：执行 `wait(mutex)` ，申请互斥资源。
    - **退出临界区**：执行 `signal(mutex)` ，释放互斥资源。
    - **阻塞条件**： `S.value < 0` 时，进程调用 `block` 自我阻塞。
    - **唤醒条件**：资源数加一后仍 `<= 0` ，说明有进程等待，应调用 `wakeup` 唤醒 `S.L` 中的第一个进程。
  - 信号量实现进程同步
    - **同步信号量**：设置 `S = 0` 。
    - **先执行动作**：需要先执行的语句后面执行 `signal(S)` 。
    - **后执行动作**：需要后执行的语句前面执行 `wait(S)` 。
    - **调度效果**：无论进程如何调度，都能保证指定的先后顺序。

by joshuji 由 幕布 发布